



Problem A. K-Color Union

Problem Setter: Raihat Zaman Nelay

Category: Trees, DP, Bitmask

Total Solved: 1

First to Solve: AUST_Simplicity

Solution:

We are given a tree with N nodes, each painted with one of K colors ($K \leq 10$). We need to count unordered pairs (u, v) whose unique path contains all K colors at least once.

Solution 1 — Tree DP with Sparse Mask Merging (Intended)

Root the tree at node 0 and perform a DFS. For each node u , maintain a *sparse map* `cache[u]`: a list of (mask, count) pairs, where mask is the set of colors occurring on a path from u to some descendant, and count is the number of such descendant nodes.

1. At node u with color c_u , initialize `cur` = $\{(1 \ll c_u, 1)\}$ — the path from u to itself.
2. For each child v of u :
 - Compute `child_map` = `cache[v]` (already computed by DFS).
 - For each entry (cmask, ccnt) in `child_map`, let `nmask` = `cmask` | $(1 \ll c_u)$.
 - **Count cross pairs:** For each (umask, ucnt) in `cur`, if `nmask` | `umask` = `allMask`, add `ccnt` × `ucnt` to the answer. These are pairs whose LCA is exactly u , using colors from different subtrees.
 - **Merge:** Insert (`nmask`, `ccnt`) into `cur`, combining entries with the same mask.
3. Store `cache[u]` = `cur`.

At the end, `ans` contains the count of all rainbow paths.

Why it works: Every pair (a, b) has a unique LCA u . When the DFS processes u , the pair is counted exactly once — a lies in one child subtree (or is u itself) and b lies in another (or is u). The mask-merge ensures we only count pairs whose combined color set covers all K colors.

Complexity: Each sparse map has at most $\min(2^K, \text{subtree_size})$ entries. The double loop over `cur` and `child_map` for each child is $\mathcal{O}(|\text{cur}| \times |\text{child_map}|)$. In the worst case (a star with $K = 10$), `cur` grows to $2^K = 1024$ and the total cross-pair work is $\mathcal{O}(N \times 2^K)$ — well within limits.

Solution 2 — DSU & Inclusion–Exclusion (Intended Alternative)

A path contains only colors from a set S iff every node on the path has a color in S . Define $f(S)$ as the number of unordered pairs (u, v) such that the path between them uses *only* colors from S .

By the inclusion–exclusion principle:

$$\text{answer} = \sum_{\substack{S \subseteq \{0, \dots, K-1\} \\ S \neq \emptyset}} (-1)^{K-|S|} \cdot f(S)$$

Computing $f(S)$: Consider only nodes whose color belongs to S . Connect two such nodes if they are adjacent in the tree and both endpoints have colors in S . The graph induced on these nodes becomes a forest. For a connected component with s nodes, any unordered pair within it has a path using only colors from S , contributing $\binom{s}{2}$ to $f(S)$.

Optimization: Bucket edges by the mask of their two endpoints' colors (`mask` = $(1 \ll c_u) \mid (1 \ll c_v)$). For



each mask S , iterate over all submasks $m \subseteq S$, and for each edge in bucket m , unite its endpoints in a DSU. This avoids scanning all edges for every S .

$$f(S) = \sum_{\text{component}} \binom{\text{size}}{2}$$

Complexity: The DSU is reset $\mathcal{O}(N)$ times, once per mask (2^K), giving $\mathcal{O}(N \times 2^K)$. The inner submask iteration totals $\mathcal{O}(3^K)$ across all masks. For $K \leq 10$, $2^K = 1024$ and $3^K = 59049$, both negligible. The dominant term is $\mathcal{O}(N \times 2^K)$ from the per-mask resets and component scans.

Problem B. Nonmetal Alchemist

Problem Setter: Irfanur Rahman Rafio

Tester: Md Mehrab Hossain Opi

Alt Writer: Raihat Zaman Nelay

Category: Combinatorics, Binary Search

Total Solved: TBD

First to Solve: BUET_MST

Each (n, ℓ, m, s) tuple contains 1 electron.

Each (n, ℓ, m) triplet contains S electrons (S valid values of s in $[1, S]$).

Each (n, ℓ) pair contains $(2\ell + 1) \times S$ electrons ($2\ell + 1$ valid values of m in $[-\ell, \ell]$).

Define the **Aufbau level** of a subshell (n, ℓ) as $x = n + \ell$.

According to the Aufbau principle, subshells are filled in increasing **Aufbau order**:

- First by increasing $x = n + \ell$
- Then by increasing n (equivalently, decreasing ℓ) when x is equal

A subshell appearing earlier in this order is filled before one appearing later.

As $0 \leq \ell < n$ and $x = n + \ell$, $n = x - \ell$, so: $0 \leq \ell < x - \ell$ which gives: $0 \leq \ell \leq \left\lfloor \frac{x-1}{2} \right\rfloor$

So, the capacity of electrons in Aufbau level x is:

$$\text{LevelCapacity}(x) = \sum_{\ell=0}^{\left\lfloor \frac{x-1}{2} \right\rfloor} (2\ell + 1) \cdot S = \left(1 + \left\lfloor \frac{x-1}{2} \right\rfloor\right)^2 \cdot S$$

Now, the total capacity of electrons in the first x Aufbau levels is:

$$\text{TotalCapacity}(x) = \sum_{i=1}^x \left(1 + \left\lfloor \frac{i-1}{2} \right\rfloor\right)^2 \cdot S$$

This can be simplified into a closed-form expression. Let $y = \left\lfloor \frac{x}{2} \right\rfloor$

Then:

$$\text{TotalCapacity}(x) = \begin{cases} \frac{y(y+1)(2y+1)}{3} \cdot S, & \text{if } x \text{ is even} \\ \left(\frac{y(y+1)(2y+1)}{3} + (y+1)^2\right) \cdot S, & \text{if } x \text{ is odd} \end{cases}$$



By re-indexing the subshells from (n, ℓ) to (x, ℓ) , an order is obtained sorted first by x ascending, then by ℓ descending. This ordering directly follows the Aufbau principle. Using this order, the **last occupied subshell** (partially or fully) can be determined via **binary search** on the Aufbau level.

Let (x, ℓ) be this last occupied subshell — i.e., the occupied subshell appearing latest in Aufbau order. Then all subshells earlier in Aufbau order are completely filled, and all subshells later in the order are empty.

Depending on the value of ℓ , the number of final shell electrons can be determined as follows:

- If $\ell = 0$: Then the shell is $n = x$. The number of final shell electrons is the number of electrons in this subshell, since electrons have only just reached this value of n .
- If $\ell = 1$: Then the shell is $n = x - 1$. It includes the electrons in the current subshell $(x - 1, 1)$, as well as all S electrons from the subshell $(x - 1, 0)$. Since that subshell appears earlier in Aufbau order, it must already be completely filled.
- If $\ell > 1$: Then $n = x - \ell < x - 1$, and the last occupied subshell belongs to a lower shell. The last time electrons were placed in a higher shell was in the subshell $(x - 1, 0)$, which must already be completely filled. So, the final shell is $n = x - 1$, and it has S electrons.

Problem C. Self Citation

Problem Setter: Tariq Hasan Rizu

Tester: Rafid Bin Mostofa

Alt Writer: Md. Irfanur Rahman Rafio, Raihat Zaman Neloy, S. M. Shaheen

Category: Greedy, Binary Search

Total Solved: TBD

First to Solve: TBD

Solution:

Let $b_1, b_2, \dots, b_k$ be the number of papers published each month. The total number of citations is

$$C = \sum_{i=2}^k b_i \cdot (b_1 + b_2 + \dots + b_{i-1}) = \sum_{1 \leq i < j \leq k} b_i b_j.$$

Since $(\sum b_i)^2 = \sum b_i^2 + 2 \sum_{i < j} b_i b_j$, we have

$$C = \frac{n^2 - \sum b_i^2}{2}.$$

With n fixed, maximizing C is equivalent to **minimizing** $\sum b_i^2$.

By convexity, for a fixed sum, the sum of squares is minimized when the b_i are as equal as possible — i.e., we want to balance the values subject to the caps a_i . This is a variant of the classic water-filling problem.

Solution idea 1: Greedy with sorting.

Sort the months by their cap a_i in ascending order. Then iterate through them. At step i (among the remaining $m = k - i$ months), let rem be the number of papers still to assign. Assign

$$b = \min(a_{\text{current}}, \lceil rem/m \rceil)$$

to this month, subtract b from rem , and continue. This gives the most balanced distribution possible because it fills the smallest caps first and distributes the rest evenly.



- Sorting: $\mathcal{O}(k \log k)$.
- Handles $k \leq 2 \times 10^6$ comfortably.

Solution idea 2: Binary search on threshold.

Find the maximum value X such that $\sum_{i=1}^k \min(a_i, X) \leq n$. This X represents the base level each month should get (capped by a_i). Binary search X in $[0, \max a_i]$.

Once X is found:

1. Set $b_i = \min(a_i, X)$ for all i .
2. Compute $rem = n - \sum b_i$.
3. Distribute the remaining rem papers one per month among those with $b_i < a_i$, starting from the last month (largest index) to produce a valid answer.

This approach avoids sorting and runs in $\mathcal{O}(k \log \max a_i)$, which is roughly $\mathcal{O}(30k)$ — very fast for the constraints.

Complexity: $\mathcal{O}(k \log k)$ or $\mathcal{O}(k \log \max a_i)$ per test case, depending on the approach.

Problem D. Floatovoltaics

Problem Setter: Tariq Hasan Rizu

Tester: Rafid Bin Mostofa

Alt Writer: Md. Irfanur Rahman Rafio, Raihat Zaman Nelay

Category: Geometry, Ternary Search, Calculus

Total Solved: TBD

First to Solve: TBD

Solution:

First, derive the cost function from the geometry.

Place CD on the x -axis with F at the origin. Let $p = CF$ and $q = DF$ (both positive), so the vertices are:

$$D = (-q, 0), \quad F = (0, 0), \quad C = (p, 0).$$

$AE \perp CD$, so AE is vertical. Let $A = (0, h)$ above CD and $E = (0, -w)$ below CD , where $w = FE$ is fixed.

1. **Right-angle condition:** $\angle CED = 90^\circ$ gives

$$\overrightarrow{CE} \cdot \overrightarrow{DE} = (p, w) \cdot (-q, w) = -pq + w^2 = 0 \implies pq = w^2 \implies q = \frac{w^2}{p}.$$

2. **Rhombus condition:** All sides are equal. Side $CD = p + q$. Side $AD = \sqrt{q^2 + h^2}$. Equating:

$$(p + q)^2 = q^2 + h^2 \implies h^2 = p^2 + 2pq = p^2 + 2w^2.$$

$$\text{Hence } h = \sqrt{p^2 + 2w^2}.$$

3. **Area:** $[ABCD] = CD \cdot h = (p + q) \cdot \sqrt{p^2 + 2w^2}$.

4. **Cable lengths:**

$$CE = \sqrt{p^2 + w^2}, \quad DE = \sqrt{q^2 + w^2}.$$



The total cost is therefore

$$\text{Cost}(p) = x \cdot (p + q) \cdot \sqrt{p^2 + 2w^2} + y \cdot (\sqrt{p^2 + w^2} + \sqrt{q^2 + w^2}),$$

with $q = w^2/p$. The function is unimodal for $p > 0$, so the minimum can be found numerically.

Solution idea 1: Ternary search on p .

Since $\text{Cost}(p)$ is convex, a ternary search over p finds the minimum. The search domain can be bounded as $p \in [\varepsilon, w^2/\varepsilon]$ for a small ε (e.g., 10^{-7}). Each evaluation of $\text{Cost}(p)$ is $\mathcal{O}(1)$. The number of iterations needed for 10^{-6} precision is about 60–80, which is fine for $t \leq 10^5$ with $\mathcal{O}(1)$ per test case.

Solution idea 2: Analytical optimization (binary search on r).

Let $r = p/w > 0$, so $p = rw$ and $q = w/r$. The cost becomes a function of r only:

$$f(r) = xw^2 \frac{r^2 + 1}{r} \sqrt{r^2 + 2} + yw \frac{r + 1}{r} \sqrt{r^2 + 1}.$$

Setting $f'(r) = 0$ leads to the optimality condition:

$$\varphi(r) = \frac{y}{xw}, \quad \text{where} \quad \varphi(r) = \frac{2(r^4 + r^2 - 1)\sqrt{r^2 + 1}}{(1 - r^3)\sqrt{r^2 + 2}}.$$

$\varphi(r)$ is strictly increasing on $(R_0, 1)$, where $R_0 = \sqrt{(\sqrt{5} - 1)/2} \approx 0.786151$ is the root of $r^4 + r^2 - 1 = 0$. For any positive $k = y/(xw)$, there is a unique $r \in (R_0, 1)$ satisfying $\varphi(r) = k$.

We binary search for r in $(R_0, 1)$. Each evaluation of $\varphi(r)$ is $\mathcal{O}(1)$, and 45–60 iterations give machine-level precision. Once r is found, evaluate $f(r)$ for the minimum cost.

This approach avoids the slower ternary search and gives more consistent precision.

Complexity: $\mathcal{O}(\log \varepsilon^{-1})$ per test case from the numerical search.

Problem E. Min-XOR MST

Problem Setter: Muhammad Muhiminul Islam Osim

Tester: Arnob Sarker

Alt Writer: Raihat Zaman Nelay

Category: Graphs, MST, Binary Lifting

Total Solved: 1

First to Solve: BRACU_Least_Short_Term_Memory

First, compute the original Minimum Spanning Tree (MST) using Kruskal's or Prim's algorithm. Let the total weight of this MST be mst . We then build the tree and use a technique like binary lifting (or Heavy-Light Decomposition) to quickly query the maximum edge weight on the path between any two nodes in $\mathcal{O}(\log N)$ time.

Since the King must cast the spell on exactly one road, changing its weight from W to $W \oplus X$, we can evaluate the new MST cost for every possible road and take the minimum. There are two cases for the chosen road:

Case 1: The road is NOT in the original MST.

If we change the weight of a non-MST edge $r = (u, v)$, its new weight is $W_r \oplus X$. It might now be cheap enough to enter the MST. Adding it forms a cycle with the original MST path between u and v . To resolve the cycle and form the new MST, we must remove the maximum weight edge on this cycle. Thus, the new cost becomes:

$$\min(mst, mst + (W_r \oplus X) - \max_edge_on_path(u, v))$$



Crucially, notice that this new cost is mathematically guaranteed to be $\leq mst$. Even if $W_r \oplus X$ is massive, we can just choose not to include r in the MST, leaving the original MST intact with cost exactly mst .

Case 2: The road IS in the original MST.

If we change the weight of an MST edge e , its new weight is $W_e \oplus X$. A naive approach might try to find a non-MST replacement edge if $W_e \oplus X$ becomes too large. However, an elegant mathematical property allows us to completely skip finding replacements!

If $W_e \oplus X$ increases, and we hypothetically replace it with some non-MST edge r crossing the cut, the new cost would be $mst - W_e + W_r$. By the cut property of MSTs, any non-MST edge r crossing the cut must have $W_r \geq W_e$. Therefore, this new cost is $\geq mst$.

But from Case 1, we know that if we just cast the spell on a non-MST edge instead, the cost is guaranteed to be $\leq mst$! Since we are looking for the absolute minimum possible cost, any replacement scenario (which yields $\geq mst$) will safely lose to the guaranteed $\leq mst$ backup plan. Thus, we don't need to compute replacements for MST edges at all. We simply evaluate the cost assuming the edge e stays in the tree:

$$mst - W_e + (W_e \oplus X)$$

(Note: If the graph is a pure tree where $M = N - 1$, there are no non-MST edges to use as replacements anyway, making this simple formula unconditionally correct.)

Summary of Algorithm:

1. Find the initial MST.
2. For each non-MST edge, use binary lifting to find the maximum edge on the tree path between its endpoints, and evaluate the cost if it replaces that edge.
3. For each MST edge, evaluate the cost if it simply stays in the tree with its new XORed weight.
4. The answer is the minimum cost found across all M edges.

The overall time complexity is $O(M \log M)$ for sorting the edges, plus $O((N + M) \log N)$ for building the binary lifting table and querying path maximums.

Problem F. Fool's Gold

Problem Setter: Irfanur Rahman Rafio

Tester: Pritom Kundu

Alt Writer: S. M. Shaheen Sha, Raihat Zaman Nelay

Category: Number Theory, Combinatorics

Total Solved: TBD

First to Solve: BUET_zeroVelocity

Start with the first condition: $\gcd(x, n) + \text{lcm}(x, n) = x + n$

Let $g = \gcd(x, n)$. Then you can write $x = gx'$ and $n = gn'$.

Now, $\gcd(x', n') = 1$ and $\text{lcm}(x, n) = gx'n'$.



Now,

$$\begin{aligned}\gcd(x, n) + \text{lcm}(x, n) &= x + n \\ \implies g + gx'n' &= gx' + gn' \\ \implies 1 + x'n' &= x' + n' \\ \implies x'n' - x' - n' + 1 &= 0 \\ \implies (x' - 1)(n' - 1) &= 0 \\ \implies x' = 1 \text{ or } n' = 1 \\ \implies x = g \text{ or } n = g\end{aligned}$$

Since the problem requires $x < n$, it is clear that x must be a divisor of n .

So the problem has been reduced to: “Count the **perfect-square divisors** of n , excluding n itself.”

Now, you can find all the divisors of n in $\mathcal{O}(\sqrt{n})$ time. If you do that, you’ll need $\mathcal{O}(t\sqrt{n})$ time for t test cases. Unfortunately for this problem, that exceeds the time limit. So you need to find a way to count the perfect-square divisors of n without searching for its divisors.

Fortunately, there is a way of enumerating the divisors of n without searching for them linearly. If you have the number n prime factorized as $n = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k}$, any divisor d has the form $d = p_1^{f_1} p_2^{f_2} \dots p_k^{f_k}$ where $0 \leq f_i \leq e_i$.

Interestingly, the problem doesn’t need you to enumerate over all divisors either. You only need to **count** the number of perfect-square divisors of n .

For d to be a perfect square, every exponent f_i must be even. So each f_i can be chosen from $0, 2, 4, \dots, 2 \lfloor \frac{e_i}{2} \rfloor$.

This gives $1 + \lfloor \frac{e_i}{2} \rfloor$ choices for the i -th prime. These choices are independent, so the total number of perfect-square divisors is $\prod_{i=1}^k \left(1 + \lfloor \frac{e_i}{2} \rfloor\right)$.

Since you need the number of perfect-square divisors of n except n itself, you need to subtract 1 if n is a perfect-square number.

Now, the remaining task is fast factorization. Store all the primes upto $N = \sqrt{10^9} < 32000$ using **Sieve of Eratosthenes**. After that, you can loop over only the prime numbers upto $\sqrt{n}$ for factorization.

Now, time complexity for factorizing n is $\mathcal{O}(\pi(\sqrt{n}))$, where $\pi(x)$ is the number of prime numbers in $[1, x]$.

After factorization, counting the perfect-square divisors takes $\mathcal{O}(\log n)$ time, which is negligible.

Time complexity for t test cases = $\mathcal{O}(t\pi(\sqrt{n}))$.

Preprocessing time complexity for Sieve = $\mathcal{O}(N \log \log(N))$.

For the given constraints, this will not exceed the time limit.

Problem G. Dynamic Ranklist

Problem Setter: Aminul Haq

Tester: Md Mehrab Hossain Opi

Alt Writer: Raihat Zaman Nelay

Category: Data Structures, DSU

Total Solved: 0

First to Solve: N/A

The main difficulty is that a team is not just a set of participants. Every merge combines the entire submission histories of two teams, and afterward the merged team must be ranked immediately. So we need one data structure for team membership, one for per-problem scoring inside a team, and one for the global scoreboard.



Use a Disjoint Set Union structure to maintain the current team of each participant. For every DSU root, store the minimum participant ID in that team, its solve count, its total penalty, and a map from problem ID to the current state of that problem. The state of one problem must remember the earliest accepted timestamp and all wrong-answer timestamps belonging to that team for that problem.

For one solved problem, let T be the minimum accepted timestamp among all submissions of the team for that problem, and let W be the number of wrong answers with timestamp strictly smaller than T . Then the contribution of that problem is $T + 20 \cdot W$. So whenever a new submission arrives, we only need to update the state of that one problem and adjust the team's total penalty by the difference between the old contribution and the new contribution.

The expensive operation is merging two teams. If you merge their problem maps naively, an adversarial sequence of merges will be too slow. The standard fix is small-to-large merging: always move the smaller problem map into the larger one. For one common problem, combine the two accepted timestamps by taking the minimum, merge the two wrong-answer sets, and recompute that problem's contribution. Because each problem entry moves only when it joins a strictly larger map, each entry is moved only $\mathcal{O}(\log Q)$ times overall.

To count how many wrong answers are before a given accepted timestamp, each problem needs an ordered set of its wrong-answer timestamps that supports insertion and counting how many timestamps are smaller than a given value. A Treap or any other balanced BST with order statistics is enough. Merging two wrong-answer sets should again be done small-to-large.

Now we still need ranks after every event. Keep one global ordered set containing only the current DSU roots, ordered by the key ($-\text{solves}$, penalty, minimum participant ID). After each submission or merge, update the affected team entries in this global ordered set. Then the team's rank is simply one plus the number of keys strictly smaller than its key in this ordering.

So the full solution is: DSU for membership, a small-to-large map of problems for each team, an ordered set of wrong-answer timestamps for each problem, and one global ordered set for ranking teams. The total complexity is $\mathcal{O}(Q \log^2 Q)$ in the standard implementation, which is easily fast enough for the given constraints, and the total memory usage is $\mathcal{O}(N + Q)$.

Problem H. Capital Logistics

Problem Setter: Yeamin Kaiser

Alt Writer: Raihat Zaman Nelay, Irfanur Rahman Rafio

Category: Trees, Binary Search, Greedy

Total Solved: TBD

First to Solve: TBD

Solution:

The convoy can only select cities that form an antichain in the tree — no selected city can be an ancestor of another on the same day. Consequently, on any root-to-leaf path, at most one city can be served per day. Over T days, at most T units can be removed from any single root-to-leaf path.

Since we can discard up to k units upfront, the question for a given T becomes: *can we discard $\leq k$ units so that every root-to-leaf path has total $\leq T$?* The answer is the smallest T for which this is possible.

Feasibility check (greedy DP).

Process the tree bottom-up. For each node u , maintain two values:

- $dp[u]$ — the maximum path sum from u down to any leaf in its subtree after optimal discarding.
- $cost[u]$ — total units discarded in the subtree of u .



Let $M = \max_{v \in \text{children}(u)} dp[v]$ be the largest child contribution.

- If $a_u + M \leq T$: the path stays within the limit, so $dp[u] = a_u + M$ and $cost[u] = \sum cost[v]$.
- If $a_u + M > T$: we must discard $(a_u + M - T)$ units from the subtree of u to cap the path sum at T . Hence $cost[u] = \sum cost[v] + (a_u + M - T)$, and $dp[u] = T$.

The check is feasible iff $cost[\text{root}] \leq k$.

Binary search.

The lower bound is 0 (if we discard everything). The upper bound is the maximum root-to-leaf sum of the original a values. Binary search on T to find the smallest feasible value.

Complexity: $\mathcal{O}(n \log \text{maxPath})$ per test case, with $\mathcal{O}(n)$ per DP evaluation.

Problem I. The Man, The Math, The Legend!

Problem Setter: Irfanur Rahman Rafio

Tester: Sharif Minhazul Islam

Alt Writer: Raihat Zaman Nelay

Category: Math, Implementation

Total Solved: TBD

First to Solve: BUET_GreyMatter

Solution:

At first, analyze a naive derivation of Rasel's winning condition:

$$\begin{aligned}
 & f(x) > 0 \\
 \implies & \frac{a}{bx+c} - d > 0 & [bx+c \neq 0] \\
 \implies & \frac{a}{bx+c} > d \\
 \implies & \frac{bx+c}{a} < \frac{1}{d} & [a \neq 0, d \neq 0, \frac{a}{bx+c} \text{ and } d \text{ have the same sign}] \\
 \implies & bx+c < \frac{a}{d} & [a > 0] \\
 \implies & bx < \frac{a-cd}{d} \\
 \implies & x < \frac{a-cd}{bd} & [b > 0]
 \end{aligned}$$

Considering all the assumptions in the derivation, you should be able to detect the following edge cases:

- If $b = 0$ and $c = 0$, $f(x)$ is always undefined, regardless of the value of x . In this case, there is no expected winner.
- If $L = R$ and $bL + c = 0$, $f(x)$ is always undefined, regardless of the value of x . In this case, there is no expected winner.
- If $L = R$, the game depends only on the value of $f(R)$. So, the winner is deterministic.
- If $b = 0$, $f(x) = \frac{a}{c} - d$ is a constant function. The winner is deterministic.
- If $a = 0$, $f(x) = -d$. The winner is deterministic.

Now, understand an **important property** of inequalities. Let $\frac{s}{t} > \frac{u}{v}$, where $s, t, u, v \neq 0$. You may be tempted to assume that $\frac{t}{s} < \frac{v}{u}$. But that is true only if either both $\frac{s}{t}$ and $\frac{u}{v}$ are positive or both are negative.



If $\frac{s}{t}$ is positive and $\frac{u}{v}$ is negative, then $\frac{t}{s} > \frac{v}{u}$ because regardless of the magnitude, $\frac{t}{s}$ is positive and $\frac{v}{u}$ is negative.

With this, you should now notice another edge case:

If $d = 0$, $f(x) = \frac{a}{bx+c}$. This will be positive if $(x > -\frac{c}{b} \text{ and } ab > 0)$ or $(x < -\frac{c}{b} \text{ and } ab < 0)$.

Since the edge cases are already checked, from this point you can assume that $a, b, d \neq 0$. As $L < R$ and x is a continuous random variable, the probability that $f(x) = 0$ exactly is 0.

Now, Rasel will win the game if $\frac{a}{bx+c} > d$. Let $g(x) = \frac{bx+c}{a}$ and define the following predicates:

$$P(x) \iff g(x) > 0$$

$$Q(x) \iff g(x) < \frac{1}{d}$$

If $d > 0$, for Rasel to win the game, both $P(x)$ **and** $Q(x)$ have to be true.

If $d < 0$, Rasel will win if either $P(x)$ **or** $Q(x)$ is true.

The **truth set** of a predicate is the set of all values in its domain that satisfy the predicate.

Let P and Q denote the truth sets of the predicates $P(x)$ and $Q(x)$ respectively, where the domain is the interval $[L, R] \subseteq \mathbb{R}$.

Since $P(x)$ and $Q(x)$ are defined by linear inequalities, their truth sets are unions of intervals.

Now,

$$P(x) \iff g(x) > 0$$

$$\implies \frac{bx+c}{a} > 0$$

$$\implies bx+c > 0 \text{ xor } a < 0$$

$$\implies bx > -c \text{ xor } a < 0$$

$$\implies x > -\frac{c}{b} \text{ xor } a < 0 \text{ xor } b < 0$$

$$\implies x > -\frac{c}{b} \text{ xor } ab < 0$$

$$Q(x) \iff g(x) < \frac{1}{d}$$

$$\implies \frac{bx+c}{a} < \frac{1}{d}$$

$$\implies bx+c < \frac{a}{d} \text{ xor } a < 0$$

$$\implies bx < \frac{a-cd}{d} \text{ xor } a < 0$$

$$\implies x < \frac{a-cd}{bd} \text{ xor } a < 0 \text{ xor } b < 0$$

$$\implies x < \frac{a-cd}{bd} \text{ xor } ab < 0$$

So, if $ab > 0$, $P = (-\frac{c}{b}, R]$ and $Q = [L, \frac{a-cd}{bd})$. And if $ab < 0$, $P = [L, -\frac{c}{b})$ and $Q = (\frac{a-cd}{bd}, R]$

Note that both P and Q must be subsets of $[L, R]$.

If two sets are represented as unions of pairwise disjoint intervals, then their intersection or union may be computed by scanning the intervals from left to right.

For intersections, every pair of overlapping intervals contributes their overlap to the resulting set; non-overlapping pairs contribute nothing.

For unions, overlapping or adjacent intervals are merged into a single larger interval, while disjoint intervals remain separate. Repeating this process for all intervals yields a new set representation of the result.

For an interval $[l, r]$, its **length** is defined as $r - l$.

More generally, the length of a union of disjoint intervals is the sum of the lengths of those intervals.



Let S be the set of all x in $[L, R]$ that lead to Rasel's victory.

$$S = \begin{cases} P \cap Q, & d > 0, \\ P \cup Q, & d < 0. \end{cases}$$

If the winning set can be expressed as $S = \bigcup_{i=1}^n [l_i, r_i]$ where $[l_i, r_i] \cap [l_j, r_j] = \emptyset$ for any $i \neq j$, then its length is

$$|S| = \sum_{i=1}^n (r_i - l_i).$$

Therefore, the winning probability is $\frac{|S|}{R - L}$.

If this probability is greater than $\frac{1}{2}$, then the expected winner is Rasel. If it is less than $\frac{1}{2}$, then the expected winner is Jewel. And if it is exactly $\frac{1}{2}$, then there is no expected winner.

Alternate Solution:

Handle the edge cases first.

Now,

$$f(x) = \frac{a}{bx + c} - d = \frac{a - bdx - cd}{bx + c}$$

Define the following predicates:

$$A(x) \iff a - bdx - cd > 0$$

$$B(x) \iff bx + c > 0$$

Simplifying the predicates yields:

$$A(x) \iff x < \frac{a - cd}{bd} \text{ xor } bd < 0$$

$$B(x) \iff x > -\frac{c}{b} \text{ xor } b < 0$$

Let A and B be the truth sets for $A(x)$ and $B(x)$ respectively. Let $A' = [L, R] - A$ and $B' = [L, R] - B$.

If $bd > 0$, $A = [L, \frac{a - cd}{bd})$ and $A' = [\frac{a - cd}{bd}, R]$. Otherwise, $A = (\frac{a - cd}{bd}, R]$ and $A' = [L, \frac{a - cd}{bd}]$.

If $b > 0$, $B = (-\frac{c}{b}, R]$ and $B' = [L, -\frac{c}{b}]$. Otherwise, $B = [L, -\frac{c}{b})$ and $B' = [-\frac{c}{b}, R]$.

Note that A , B , A' and B' must be subsets of $[L, R]$.

After determining A , B , A' and B' , you can easily determine $S = (A \cap B) \cup (A' \cap B')$.

Alternate Solution:

Handle the edge cases first.

Now, notice that the truth values of $A(x)$ and $B(x)$ only change at two values of x .

For $A(x)$, the change happens at $x = \frac{a - cd}{bd}$. For $B(x)$, the change happens at $x = -\frac{c}{b}$.

These two values divide the interval $[L, R]$ into at most 4 pairwise disjoint **partitions**. Inside each partition, the sign of $f(x) = \frac{a - bdx - cd}{bx + c}$ remains fixed for all x throughout the whole partition.



So instead of working with interval intersections or unions, you can process every partition independently.

Take the values L , R , $-\frac{c}{b}$, and $\frac{a - cd}{bd}$. Keep only the values inside $[L, R]$, then sort them. These values define the **borders** of the partitions.

For each partition $[l, r]$, pick any x inside it. The midpoint $x = \frac{l + r}{2}$ is convenient. Then evaluate $f(x)$:

- If $f(x) > 0$, Rasel wins for the whole partition, so add $r - l$ to Rasel's total winning set length.
- If $f(x) < 0$, Jewel wins for the whole partition, so add $r - l$ to Jewel's total winning set length.
- If $f(x)$ is undefined, ignore that partition.

After processing all partitions, compare the total winning set length of Rasel and Jewel and determine the expected winner.

Problem J. Resonance Frog

Problem Setter: Mohidul Haque Mridul

Tester: Irfanur Rahman Rafio

Alt Writer: Irfanur Rahman Rafio

Category: Dynamic Programming, Data Structures

Total Solved: TBD

First to Solve: RUET_Jyotichashi

Observation 1: We want to find the minimum number of jumps to reach pad n . At any pad i , the decisions we make only depend on our current position and the frequencies, which are static. The problem exhibits optimal substructure. Furthermore, the actual value of a frequency does not matter, only whether two frequencies are equal.

Hint 1: Coordinate compress the frequencies to a smaller range. Define $dp[i]$ as the minimum number of jumps required to reach pad i . The base case is $dp[1] = 0$, and all other $dp[i] = \infty$.

Observation 2: Jumps of different tunes do not affect the DP values of each other for action 3 (the mid-air drop). A frog can only drop onto pad k if its active Launch Tune matches the pad's Echo Frequency (b_k).

Hint 2: Because tunes are independent for action 3, we can independently manage and query the best active leap flying over the current pad for each unique frequency.

Observation 3: When singing Launch Tune a_i at pad i , the previous occurrence j where $b_j = a_i$ can be found deterministically (it is simply the most recent one). Therefore, each pad i generates at most one valid leap range.

Hint 3: Combined across all frequencies, there are at most N valid ranges for leaps in the entire problem. This strictly bounds the total number of intervals we have to process.

Observation 4: When processing pad k , multiple leaps of frequency b_k might be flying over it. However, we only care about the optimal leap.

Hint 4: Only the minimum of all valid leaps flying over the current pad of that frequency matters. This is essentially a point query for the minimum over many active intervals.

Solution: We can always transition to $dp[i + 1] = \min(dp[i + 1], dp[i] + 1)$ directly.

To handle leaps efficiently:

1. Maintain a min-priority queue for each frequency to handle the active intervals. The queue stores active leaps as pairs of (jumps taken, expiration pad m).



2. When processing pad k , we lazily remove expired leaps (where $m \leq k$) from the priority queue for b_k . If the queue is not empty, the top element represents the optimal active leap we can drop from. We update $dp[k] = \min(dp[k], \text{top jumps} + 1)$.
3. If there is a valid previous occurrence j for the Launch Tune a_k , we compute the new destination $m = 2k - j$. We directly update $dp[m] = \min(dp[m], dp[k] + 1)$ and push $(dp[k], m)$ into the priority queue for a_k to handle future drops.

Because there are at most N total ranges, the priority queues will handle at most N pushes and N pops. This yields an elegant $O(N \log N)$ solution.

Problem K. Diagonal Shortcuts

Problem Setter: Tariq Hasan Rizu

Tester: Rafid Bin Mostofa, Raihat Zaman Nelay

Alt Writer: Md. Irfanur Rahman Rafio, Rafid Bin Mostofa

Category: Math, Ad-hoc

Total Solved: TBD

First to Solve: TBD

Solution:

We have two options: use only knight moves (each costing 1), or use the special jump (costing k). The answer is $\min(k, f(x))$, where $f(x)$ is the minimum number of knight moves needed to reach (x, x) from $(0, 0)$.

Finding $f(x)$.

On an infinite chessboard, knight moves always change the parity of the sum of coordinates (a knight moves 1 step along one axis and 2 along the other, so the sum changes by $\pm 1 \pm 2 = \pm 3$ or $\pm 1 \pm 2 = \pm 1$, both odd). Since $(0, 0)$ has an even sum, (x, x) also has an even sum $2x$, so $f(x)$ must be even.

The problem of minimizing knight moves to reach (x, x) is a classic one. The minimum number follows a simple closed form:

$$f(x) = \begin{cases} 0, & x = 0, \\ 2, & x = 1, \\ 4, & x = 2, \\ 2 \cdot \left\lceil \frac{x}{3} \right\rceil, & x \geq 3. \end{cases}$$

Equivalently, for $x \geq 1$ and $x \neq 2$:

$$f(x) = 2 \cdot \left\lceil \frac{x+2}{3} \right\rceil.$$

Why this formula?

Each knight move changes both x and y coordinates, and to reach a point on the diagonal $y = x$, we need roughly $2x/3$ moves because the total Manhattan-like progress toward the diagonal is limited per move. The sequence of minimum moves for $x = 1, 2, 3, \dots$ is:

2, 4, 2, 4, 4, 4, 6, 6, 6, 8, 8, 8, ...

Two exceptions stand out:



- $x = 1$: 2 moves are needed, e.g., $(0, 0) \rightarrow (-1, 2) \rightarrow (1, 1)$.
- $x = 2$: 4 moves are required (the shortest path cannot be done in 2 moves, and 3 moves is impossible due to parity). The formula $2 \cdot \lceil x/3 \rceil$ would give 2, so this is handled as a special case.

For $x \geq 3$, the formula $2 \cdot \lceil x/3 \rceil$ holds—the value increases by 2 every third integer.

Complexity: $\mathcal{O}(1)$ per test case.

Problem L. Onion Peeling

Problem Setter: Raihat Zaman Nelay
Category: Computational Geometry
Total Solved: 0
First to Solve: N/A

Solution:

This problem asks us to compute the convex layers (onion peeling) of a set of N points and then answer point-in-layer queries. The naive approach of recomputing the convex hull from scratch each iteration costs $\mathcal{O}(N^2 \log N)$, which is too slow for $N = 5 \times 10^4$.

Key observation: After sorting the points once by (x, y) , we can maintain the active set as a doubly linked list. Removing hull vertices is $\mathcal{O}(H)$ per layer, and scanning the active set to build each hull is $\mathcal{O}(\text{active_size})$.

1. Sort points by (x, y) .
2. Build a doubly linked list (prev / next pointers) over the sorted order.
3. Repeatedly compute the convex hull of the active points:
 - Lower hull: scan left to right via **next** pointers, maintaining the lower chain.
 - Upper hull: scan right to left via **prev** pointers, maintaining the upper chain.
4. Remove hull vertices from the linked list — they are skipped in future iterations.
5. Stop when fewer than 3 points remain.

Each point survives until the layer it belongs to is peeled. Once a point is on a hull, it is removed in $\mathcal{O}(1)$ via the linked list. The total amount of work across all layers is:

$$\text{scanned} = \sum_{\ell=1}^L |\text{active}_{\ell}| \approx N + (N - H_1) + (N - H_1 - H_2) + \dots$$

In the worst case (nested triangles), $H_{\ell} = 3$ for every layer, so $L \approx N/3$ and the total scanned points is $\approx N^2/6$. With $N = 5 \times 10^4$, this is $\approx 4 \times 10^8$ operations, which runs comfortably under the 15-second time limit with an optimized $\mathcal{O}(1)$ monotone chain step per scan.

Answering queries: Store each layer as a convex polygon in CCW order (already obtained from the hull construction). For a query point P , binary search over layers:

$$\text{lo} = 0, \quad \text{hi} = L$$

and test the midpoint using a point-in-convex-polygon test in $\mathcal{O}(\log H)$ — checking orientation against the two extreme edges and binary searching the angular order. The innermost layer that strictly contains P determines the answer.

Complexity:



- Peeling: $\mathcal{O}(N \log N + \sum |\text{active}_\ell|)$.
- Query: $\mathcal{O}(Q \log L \log H)$ — each step of the binary search over layers calls a $\mathcal{O}(\log H)$ point-in-polygon test. With $Q \leq 10^6$, the total is still acceptable because L is bounded by N and each test is logarithmic.